



Ingress

dynamically expose applications to outside world

Fabrice Jammes, Karim Ammous, <https://k8s-school.fr>



<https://k8s-school.fr>

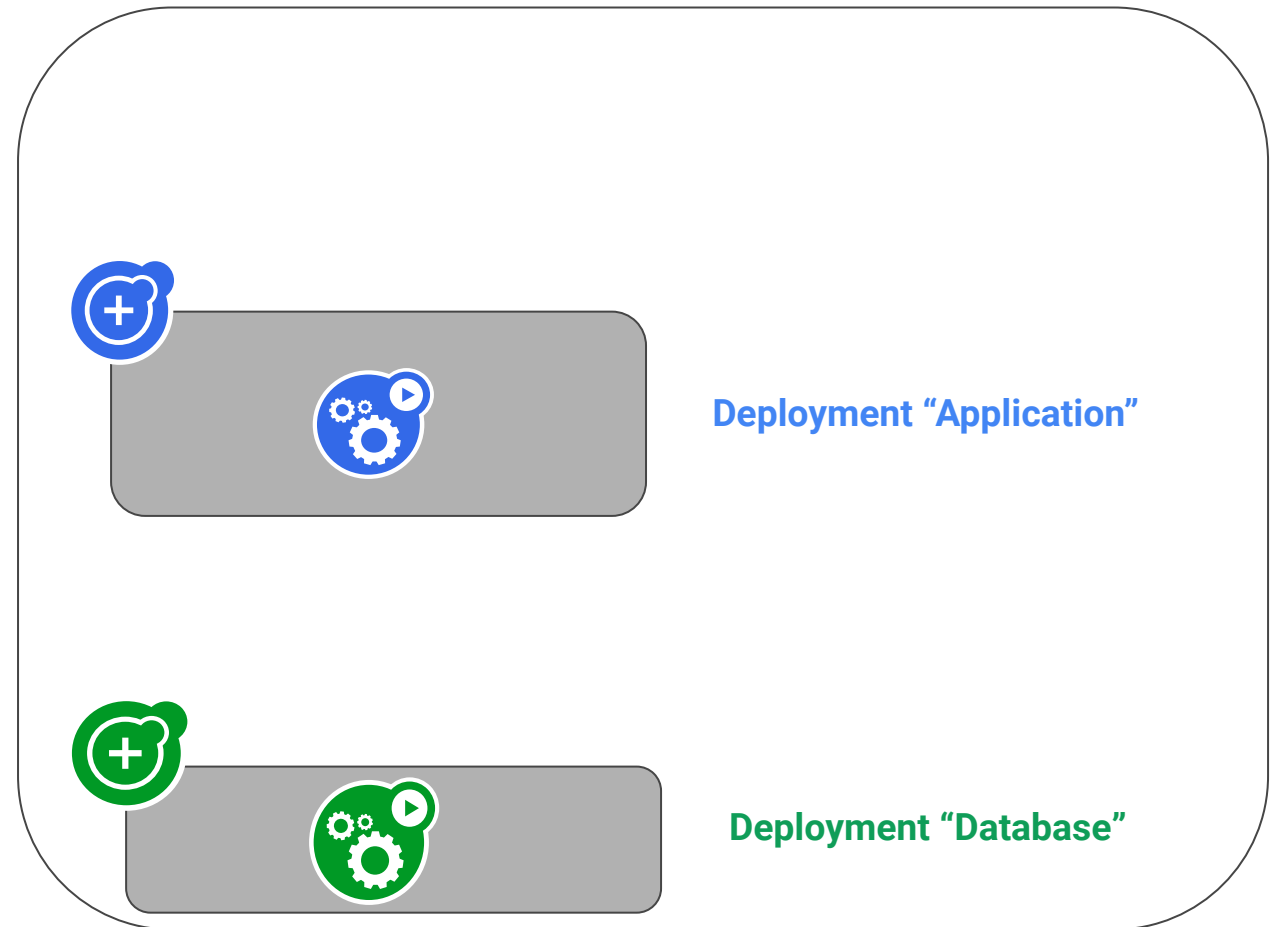
Expose application to outside world

Create application

<http://www.application.com>

Create two deployments

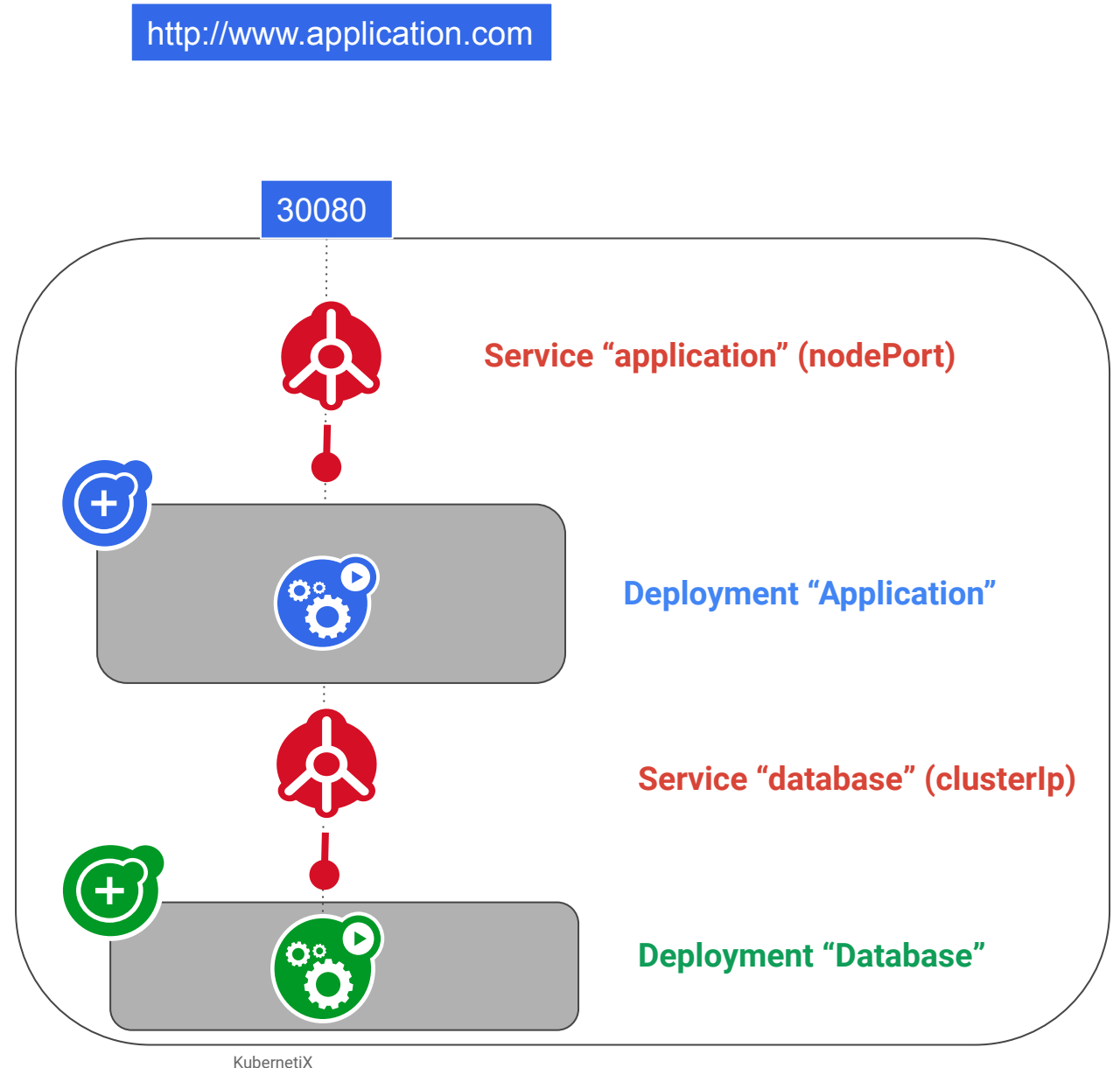
- one for a web application
- one for its related database



Create services

Create two services

- one for a web application
- one for its related database



Basic setup

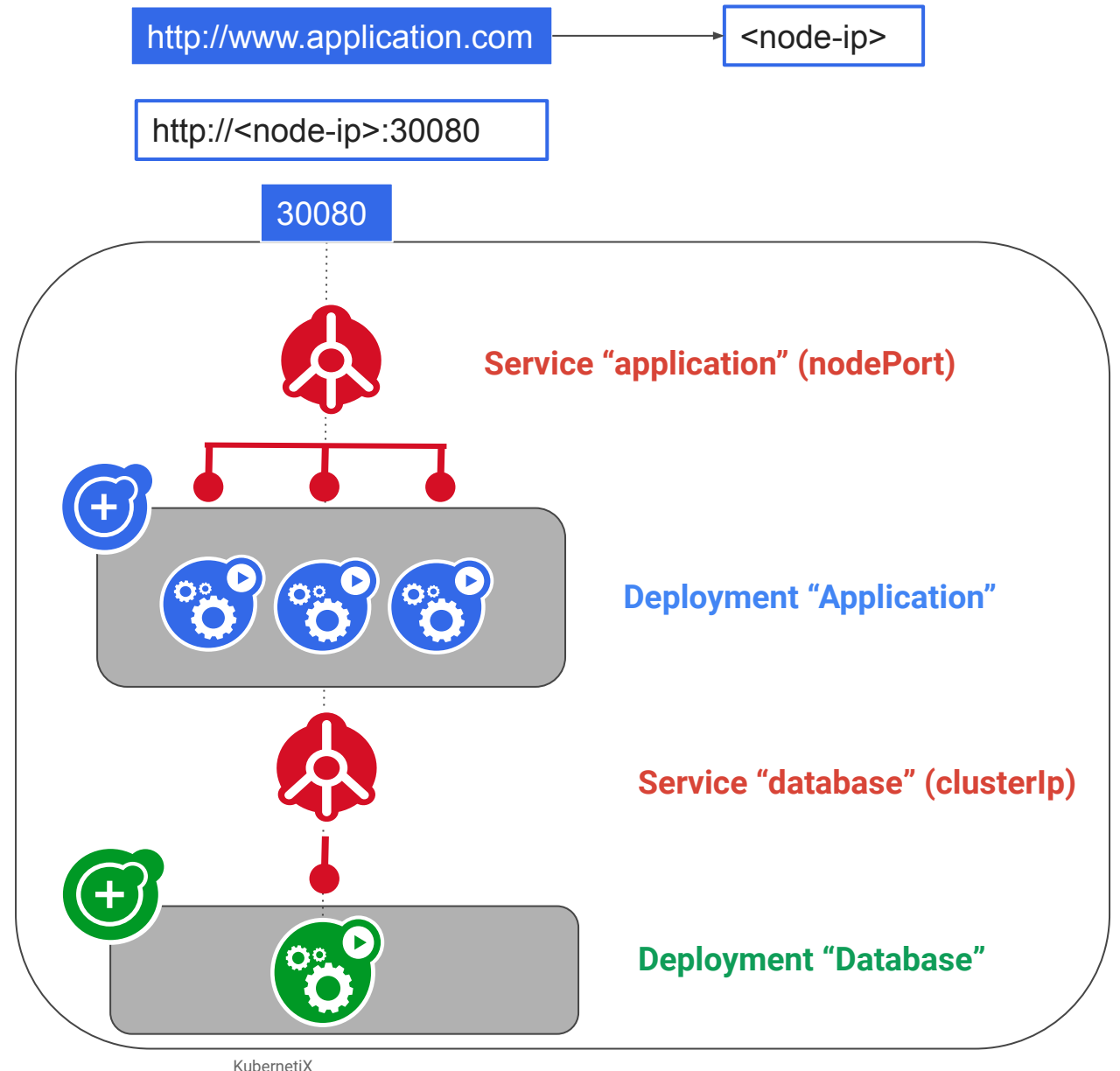
Redirect the application url to <node-ip> using DNS server

Pro

- easy to set-up

Cons

- no load balancing
- user must type the nodePort number in the URL
- Node is exposed to outside world



On premise

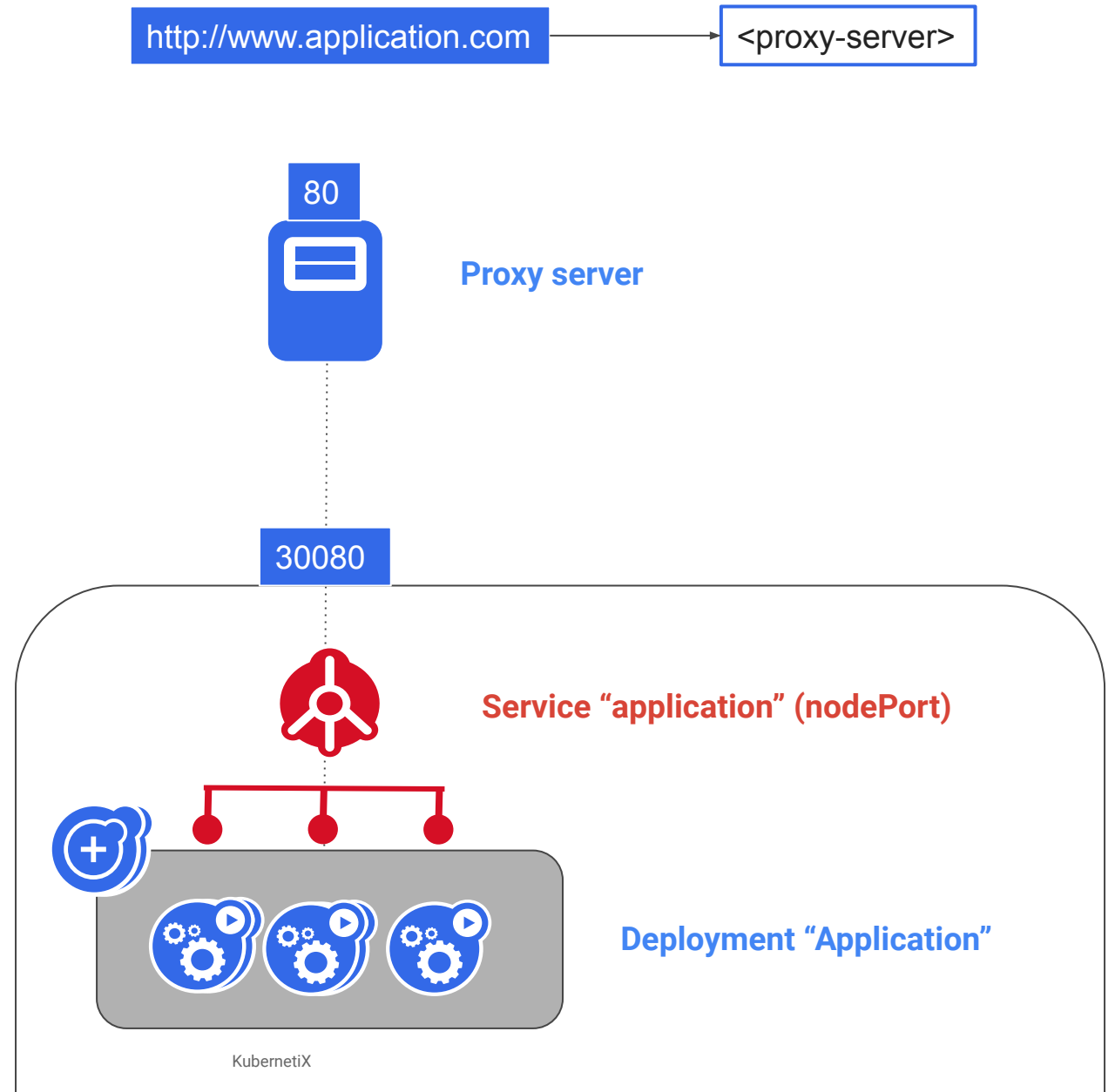
Use a proxy server

Pro

- easy to set-up
- enable load-balancing

Cons

- must be done each time a new application is exposed



In the public cloud

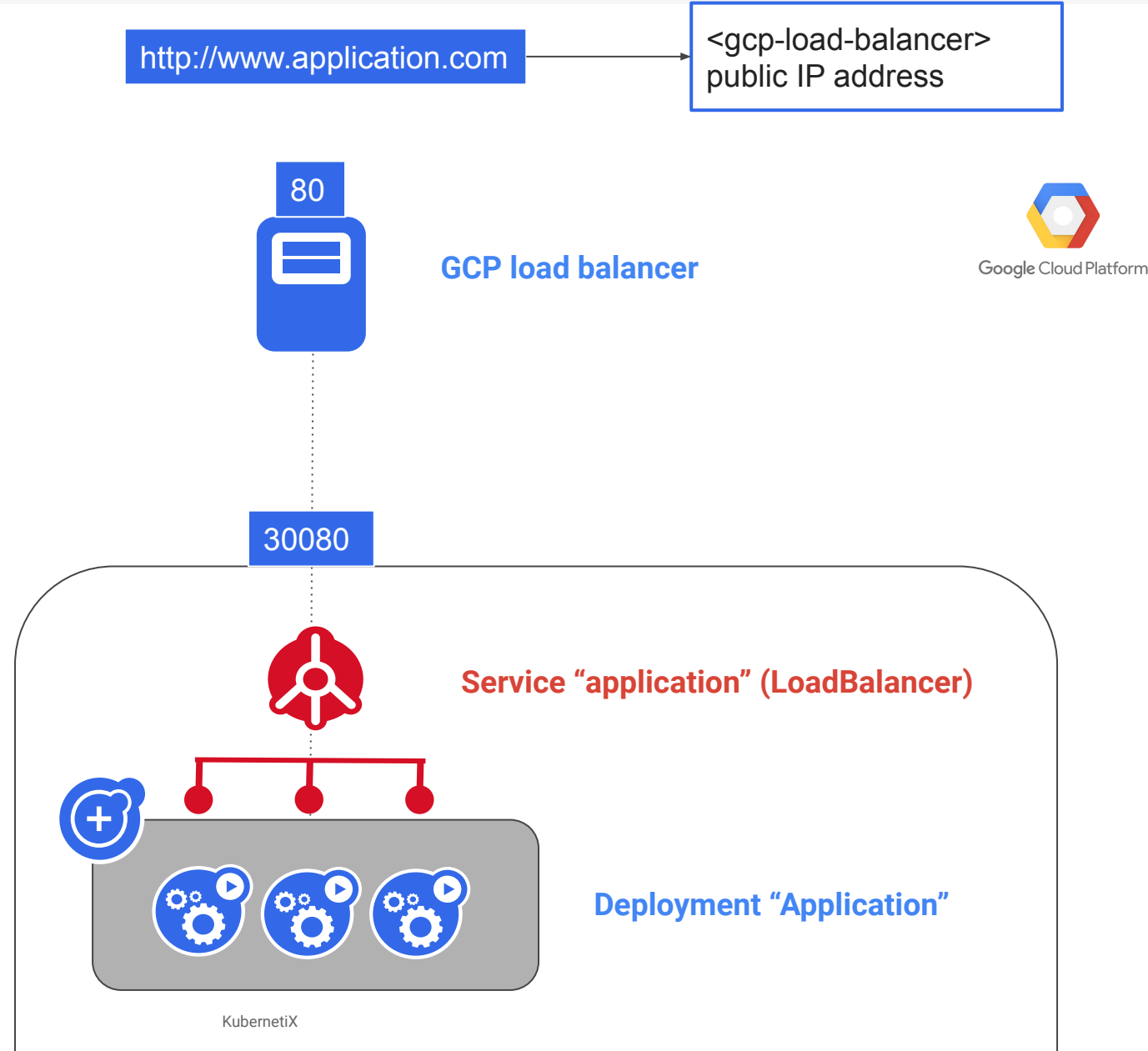
Use a “Load Balancer” service

Pro

- cloud provider does all the job

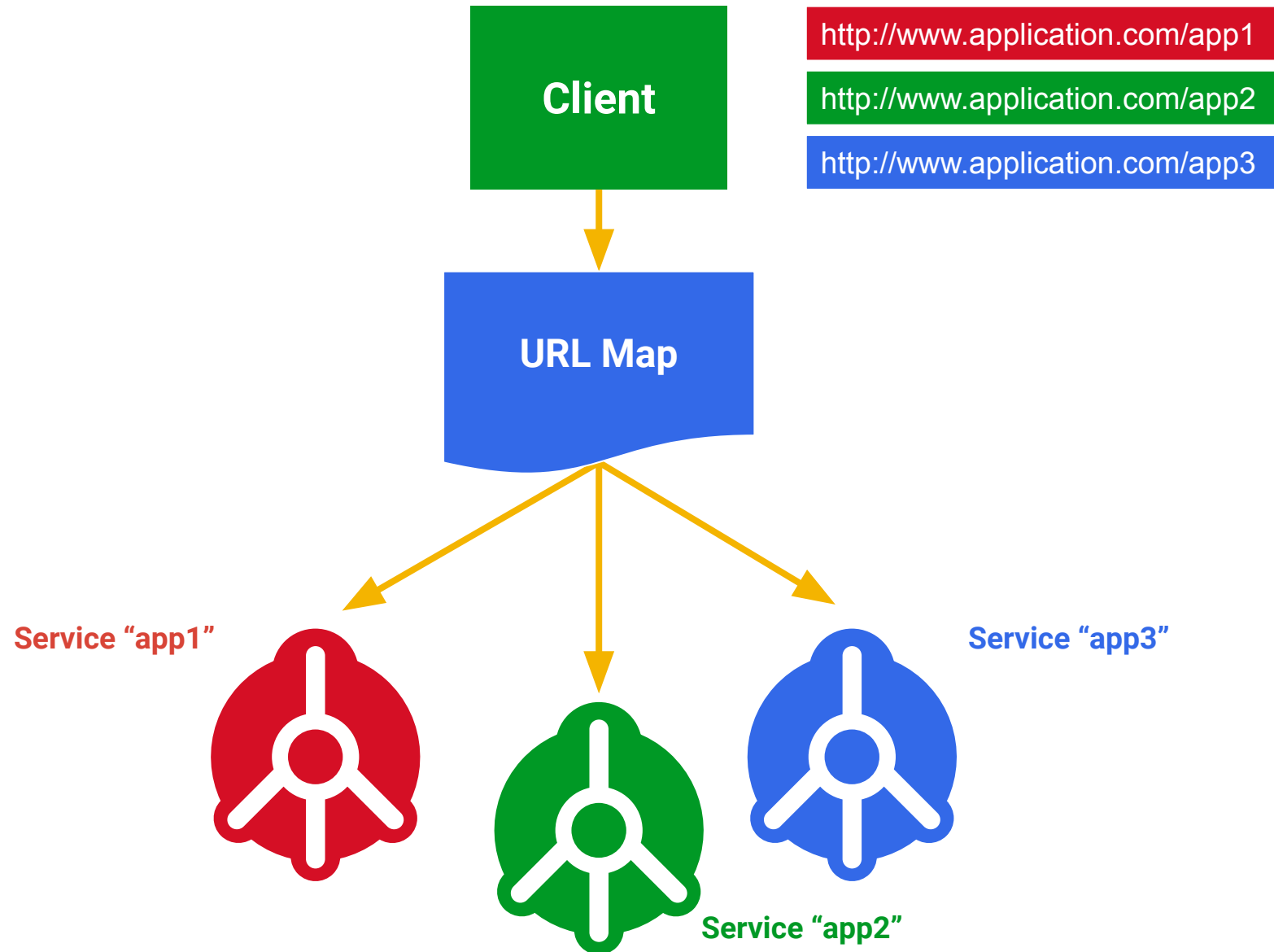
Cons

- public IP addresses may be expensive



Expose multiple applications

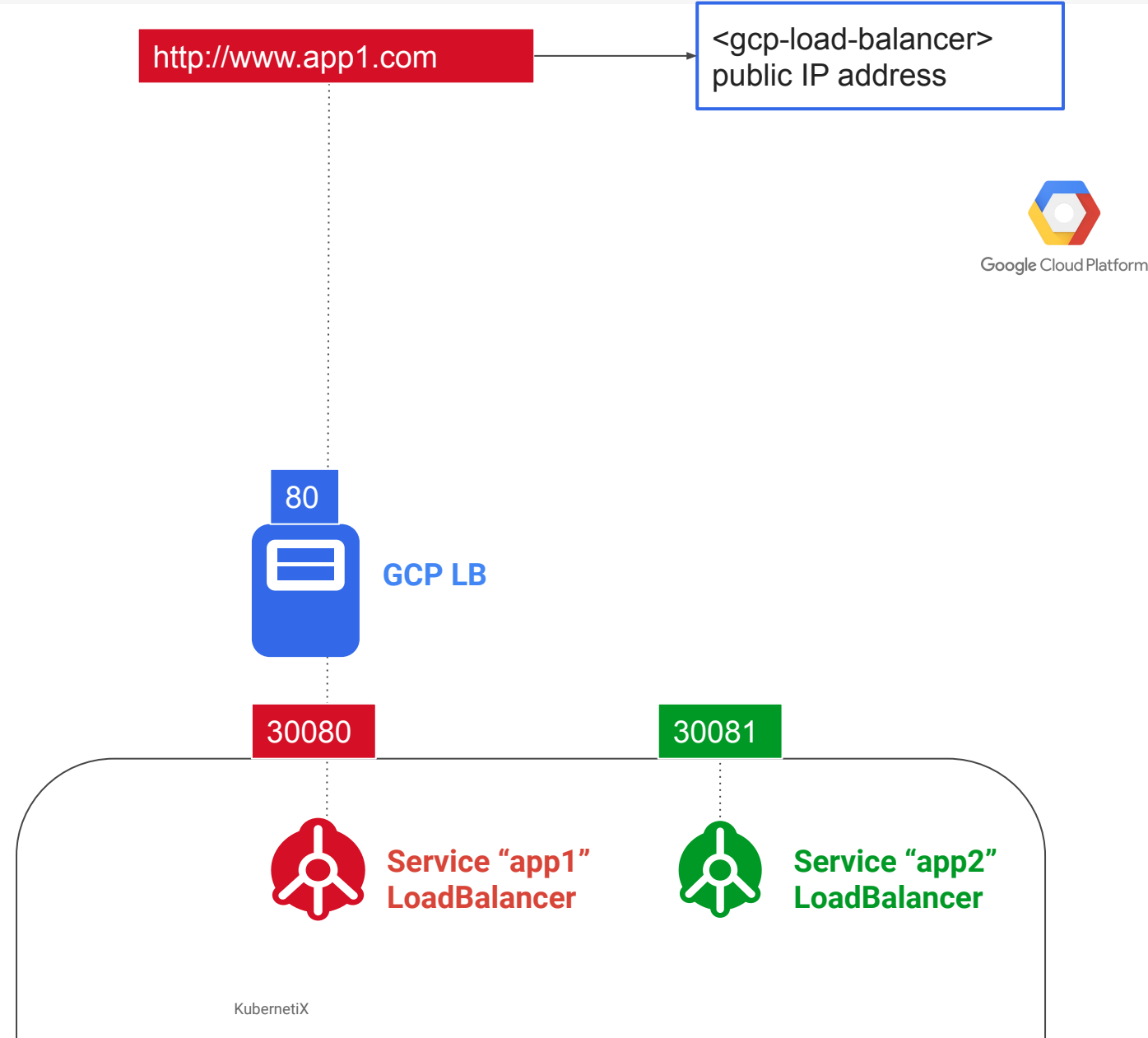
Requirement



In the public cloud

Question:

How to expose a second application?



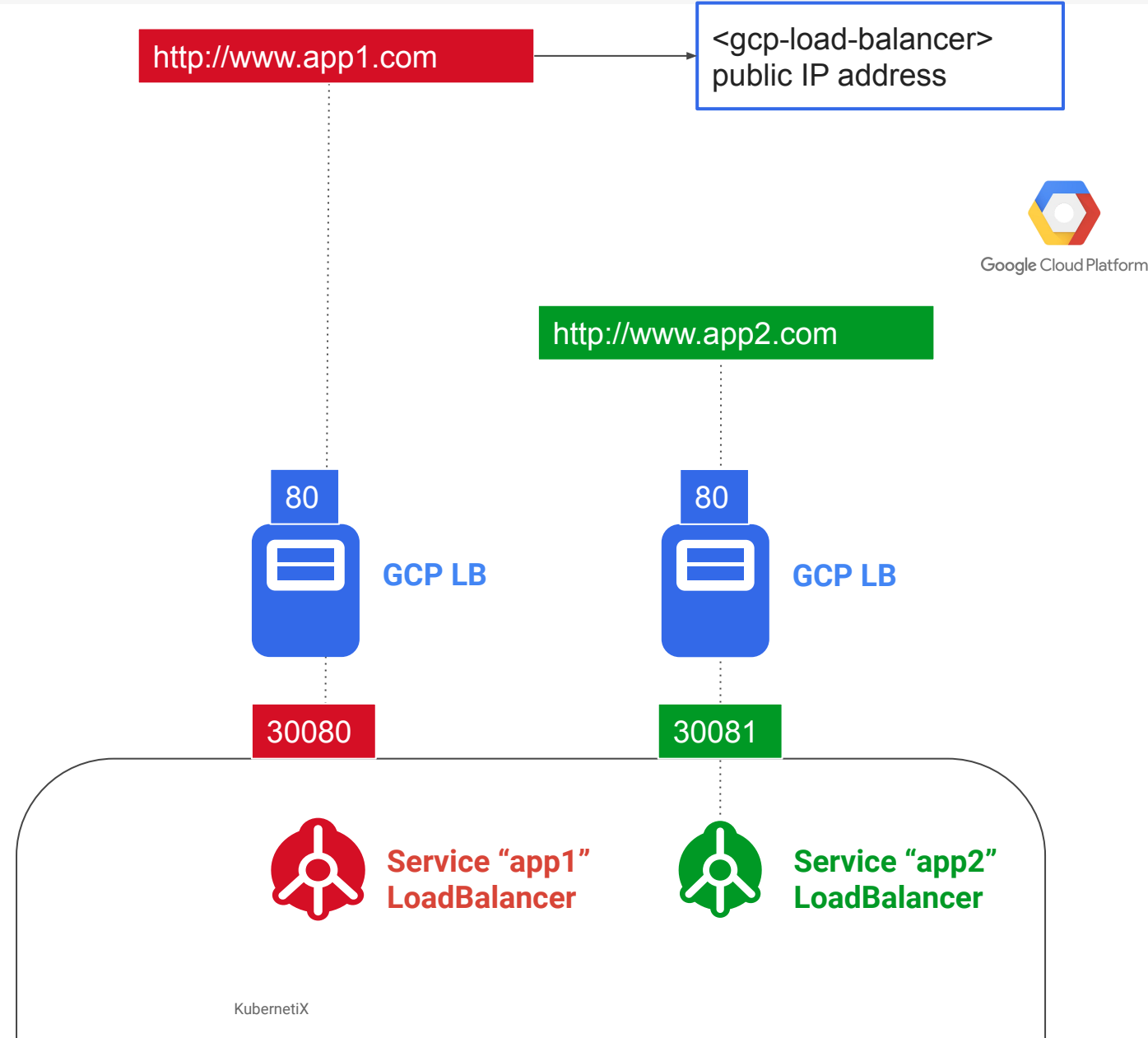
In the public cloud

Answer :

Use a second “Load Balancer”
service

Cons

- Use a second public IP address



In the public cloud

Question :

How to expose both applications
using the same host name
“www.application.com”

<http://www.application.com/app1>

<http://www.application.com/app2>



Google Cloud Platform

?

30080



Service “app1”
LoadBalancer

30081



Service “app2”
LoadBalancer

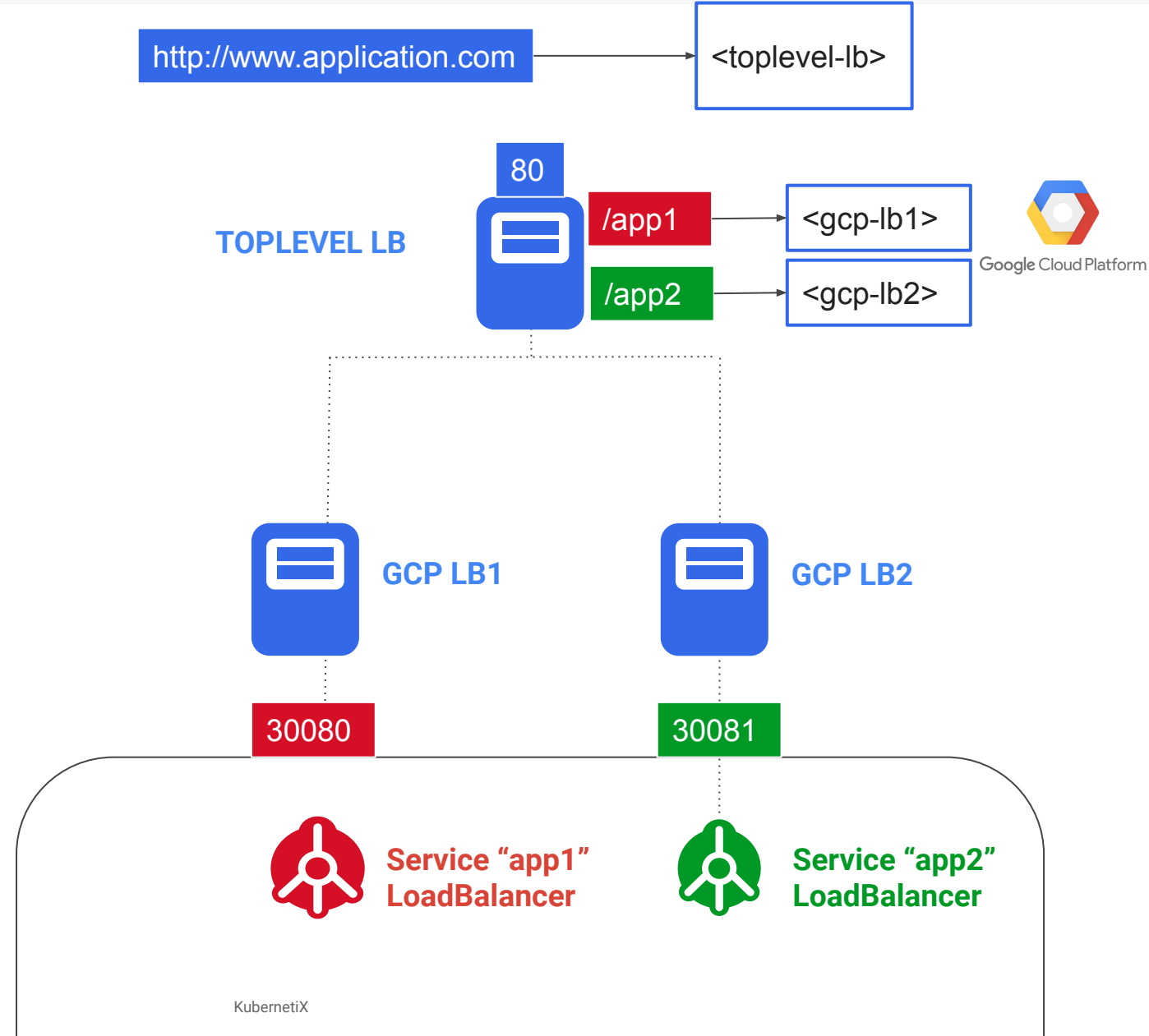
In the public cloud

Answer :

Create an other proxy server

Cons

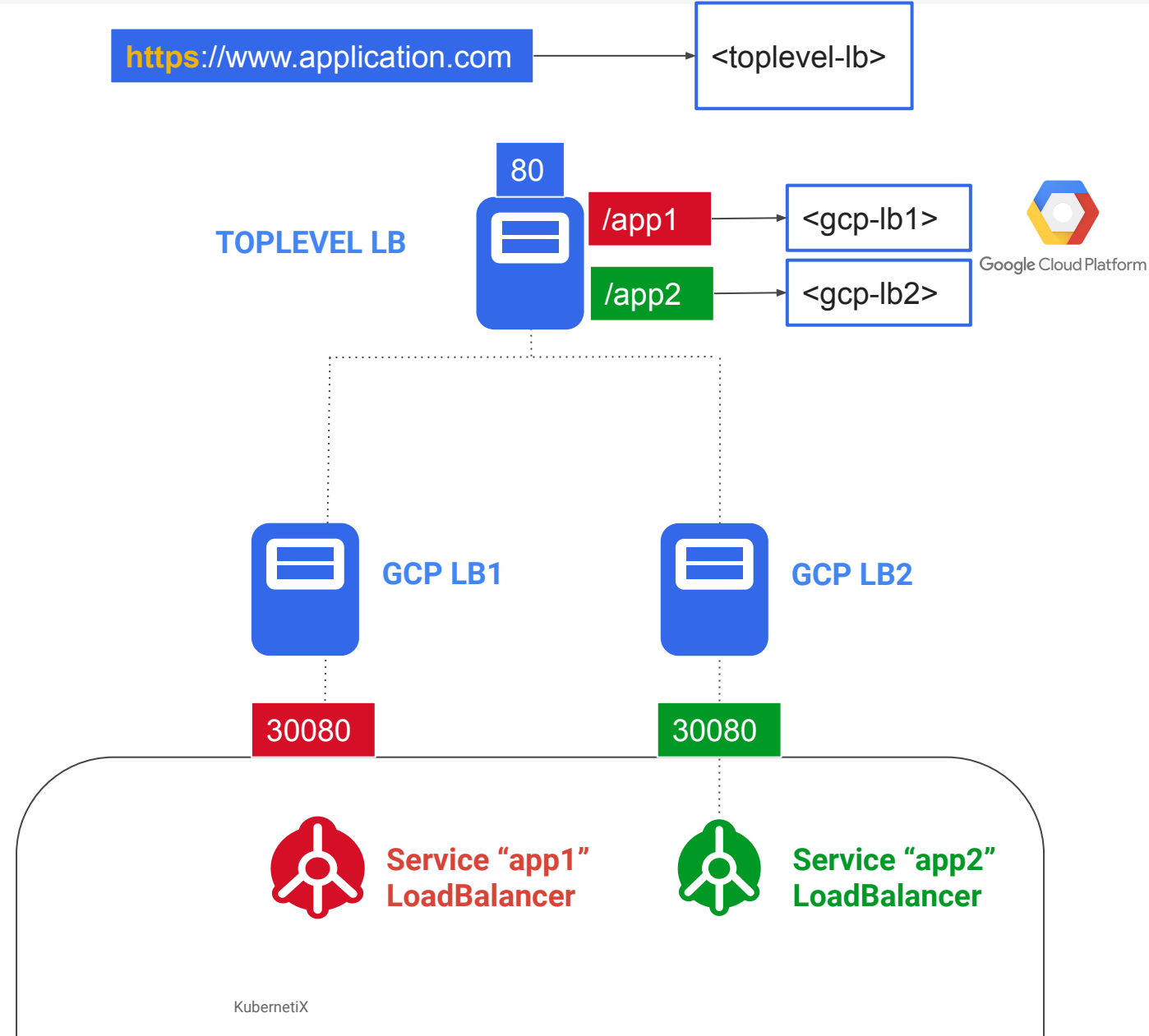
- Complicated
- Expensive
- Maintenance



In the public cloud

Question :

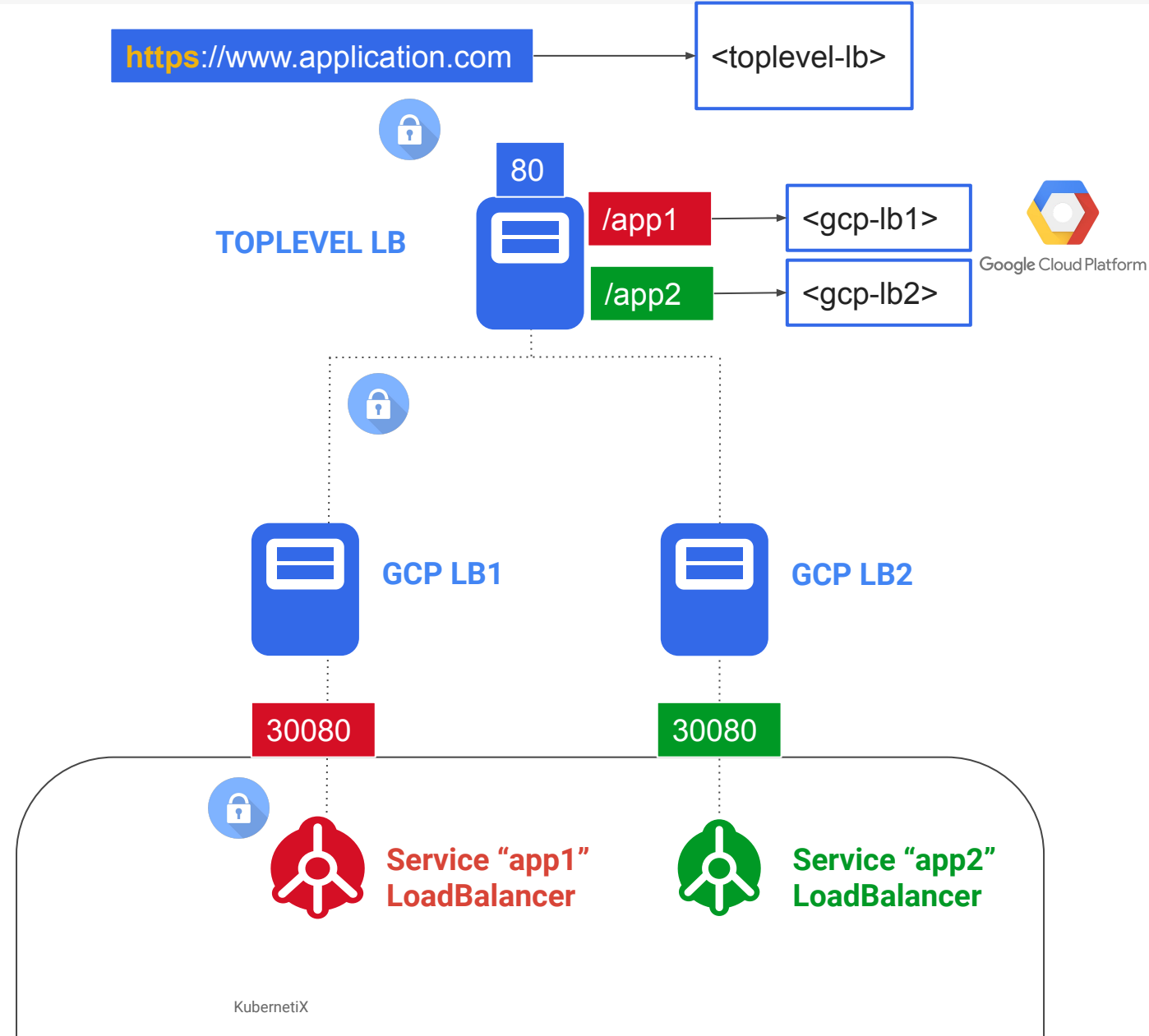
How to add https?



In the public cloud

- At toplevel lb level?
- At secondary LB level?
- At application level?

=> Exposing applications to outside world is complex in DIY mode



Ingress

Ingress (L7 LB)

Many apps are HTTP/HTTPS

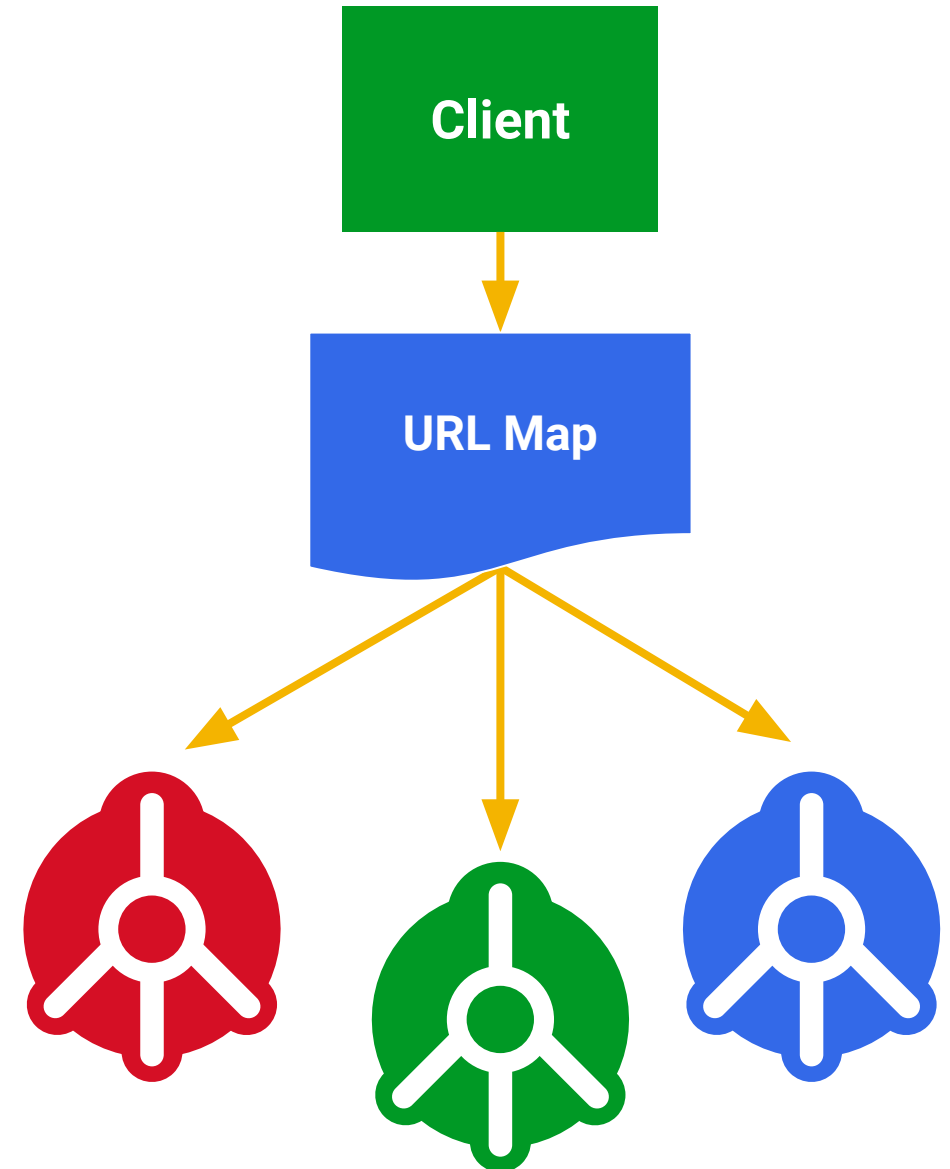
Services are L4 (IP + port)

Ingress maps incoming traffic to backend services

- by HTTP host headers
- by HTTP URL paths

HAProxy, NGINX, AWS and GCE implementations

SSL support



Implementation details

Ingress is composed of resources and controller

- Resources are rules and are part of k8s standard API
- Controller ensure these rules are enforced

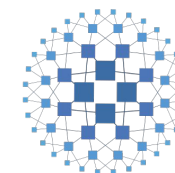
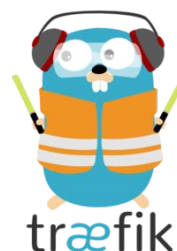


Ingress controllers

Top ingress controllers

- Contour
- Kong
- Traefik (easy)
- Nginx
- Istio ingress controller
- Haproxy (load balancing, robust)
- F5 ingress controller
- ...

See <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>



HAPROXY



Google Cloud Platform

GCP Load Balancer

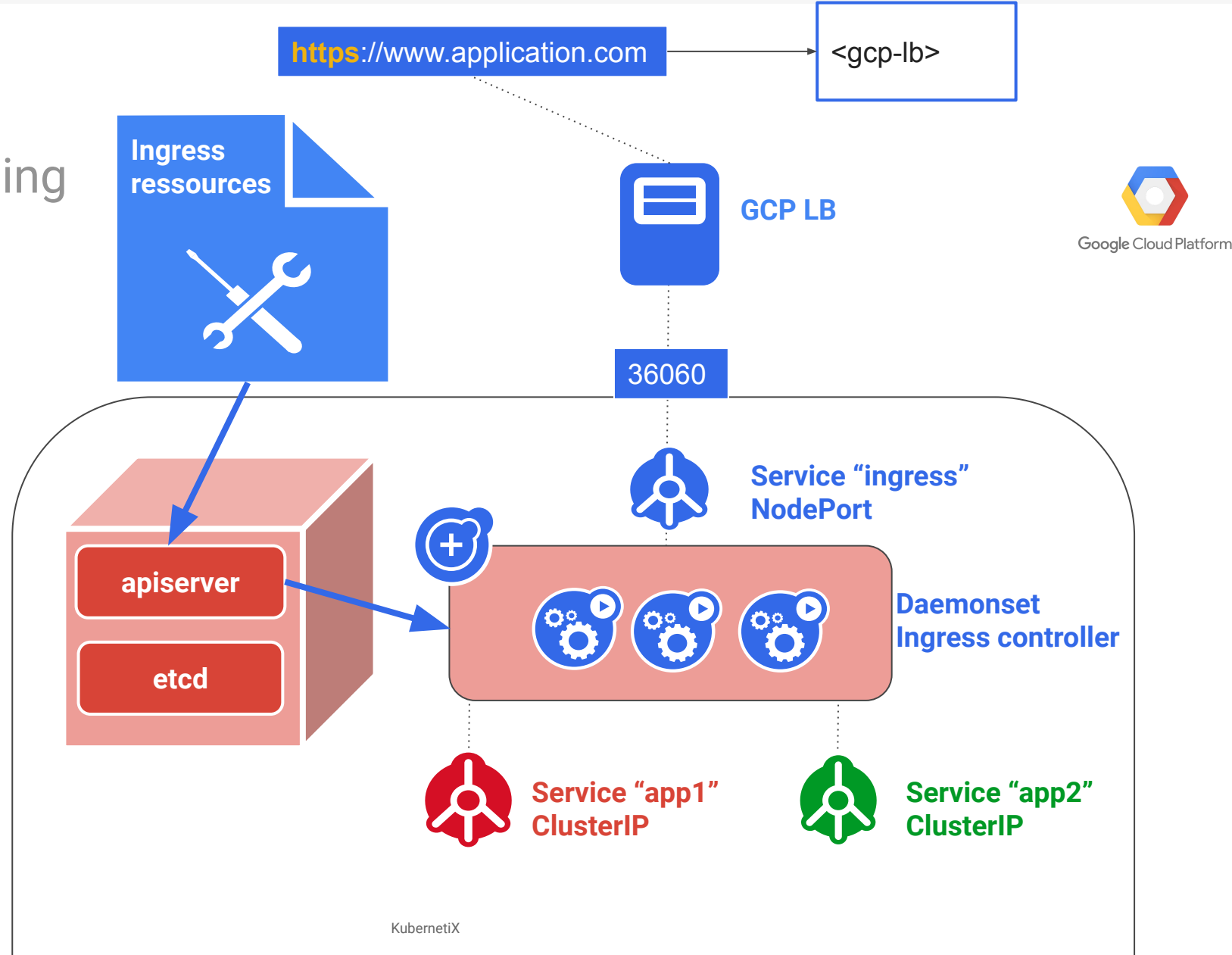
How it works

Ingress resources

- Contains a list of rules matched against all incoming requests.
- Only supports rules for directing HTTP traffic.

Ingress controller

- Only creating an Ingress resource has no effect
- Mandatory to satisfy an Ingress



On-premise: traefik example

Add your static load balancer

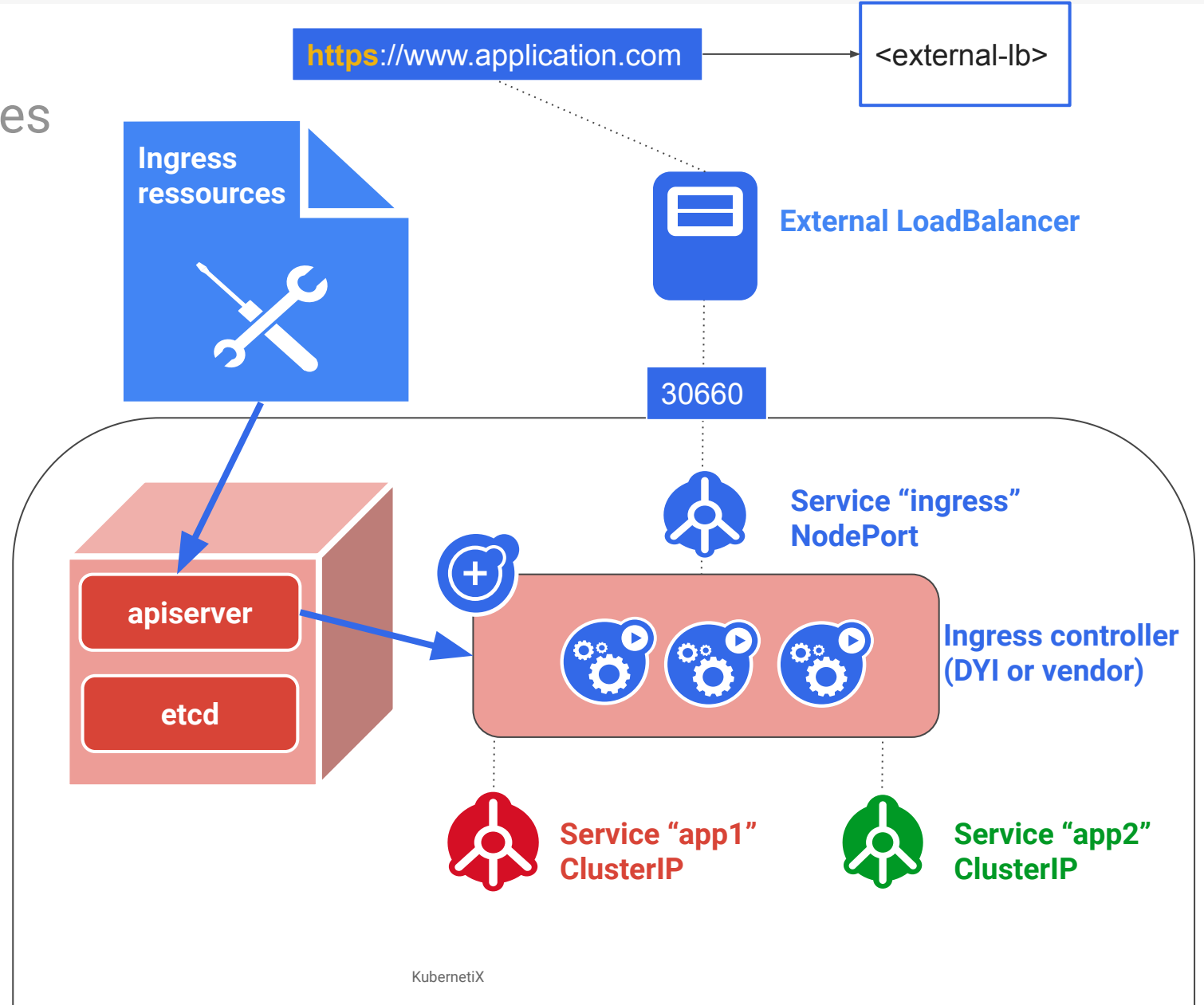
- Trivial configuration: all goes to traefik node port
- Traefik will do the internal routing

Pro:

- No write access to external load balancer configuration required

Cons:

- Not so efficient



On-premise

Add your external dynamic load balancer

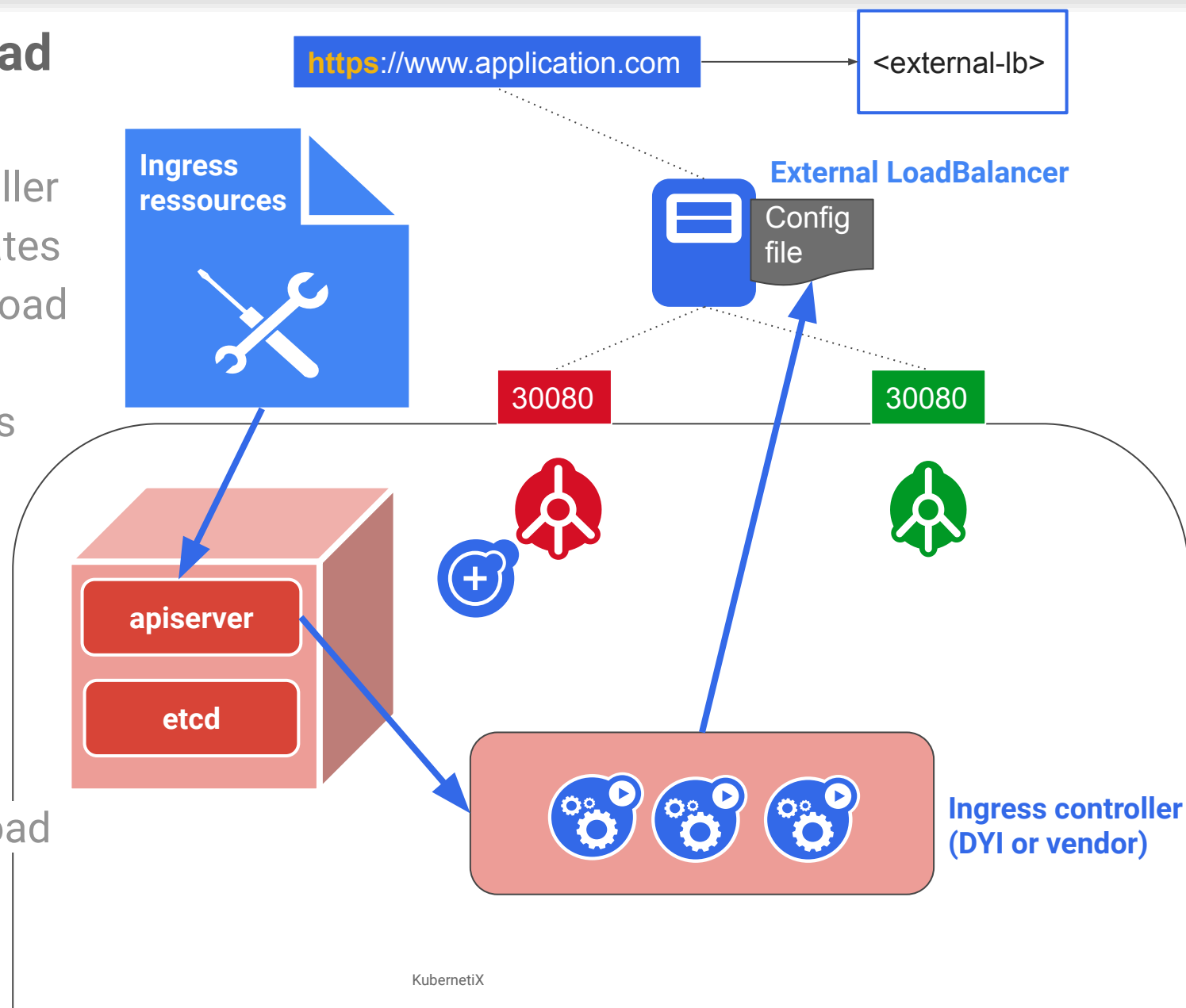
- DIY: create a k8s ingress controller which reads apiserver and updates configuration files on external Load Balancer
- Appliance: Install vendor ingress controller which will do the job

Pro:

- Efficiency
- Pretty simple

Cons:

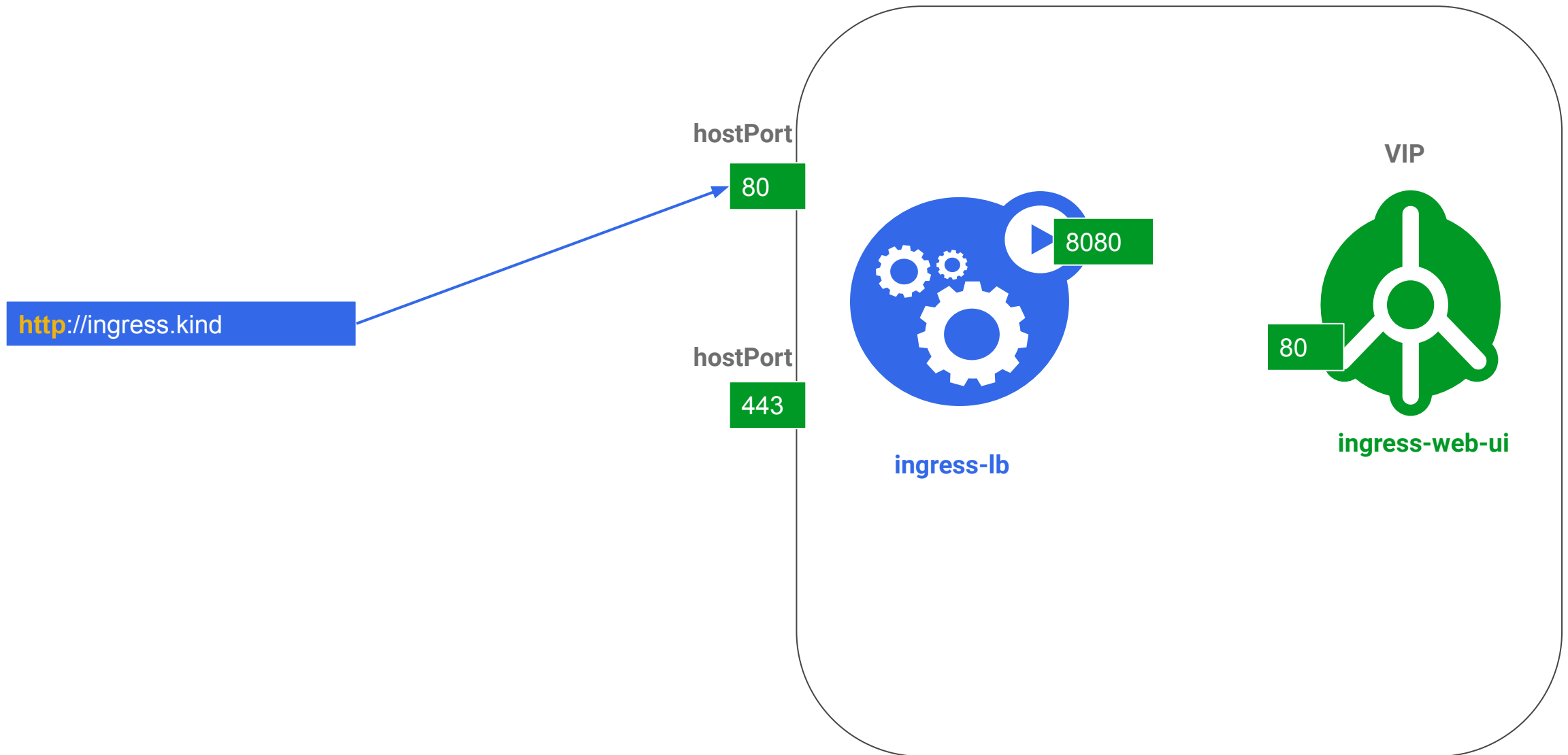
- Read write access to external load balancer configuration



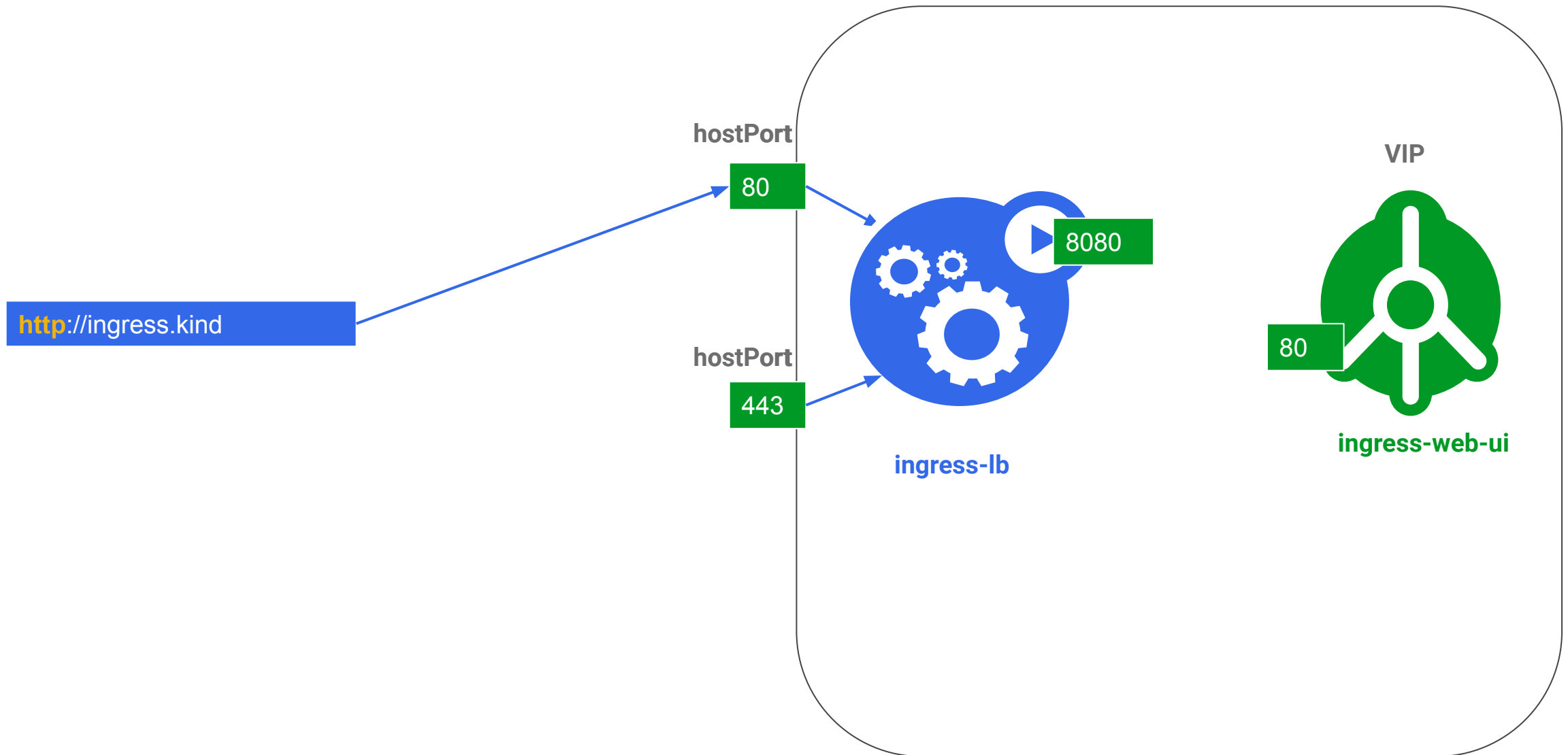
Demo: traefik

<https://docs.traefik.io/v1.7/user-guide/kubernetes/>

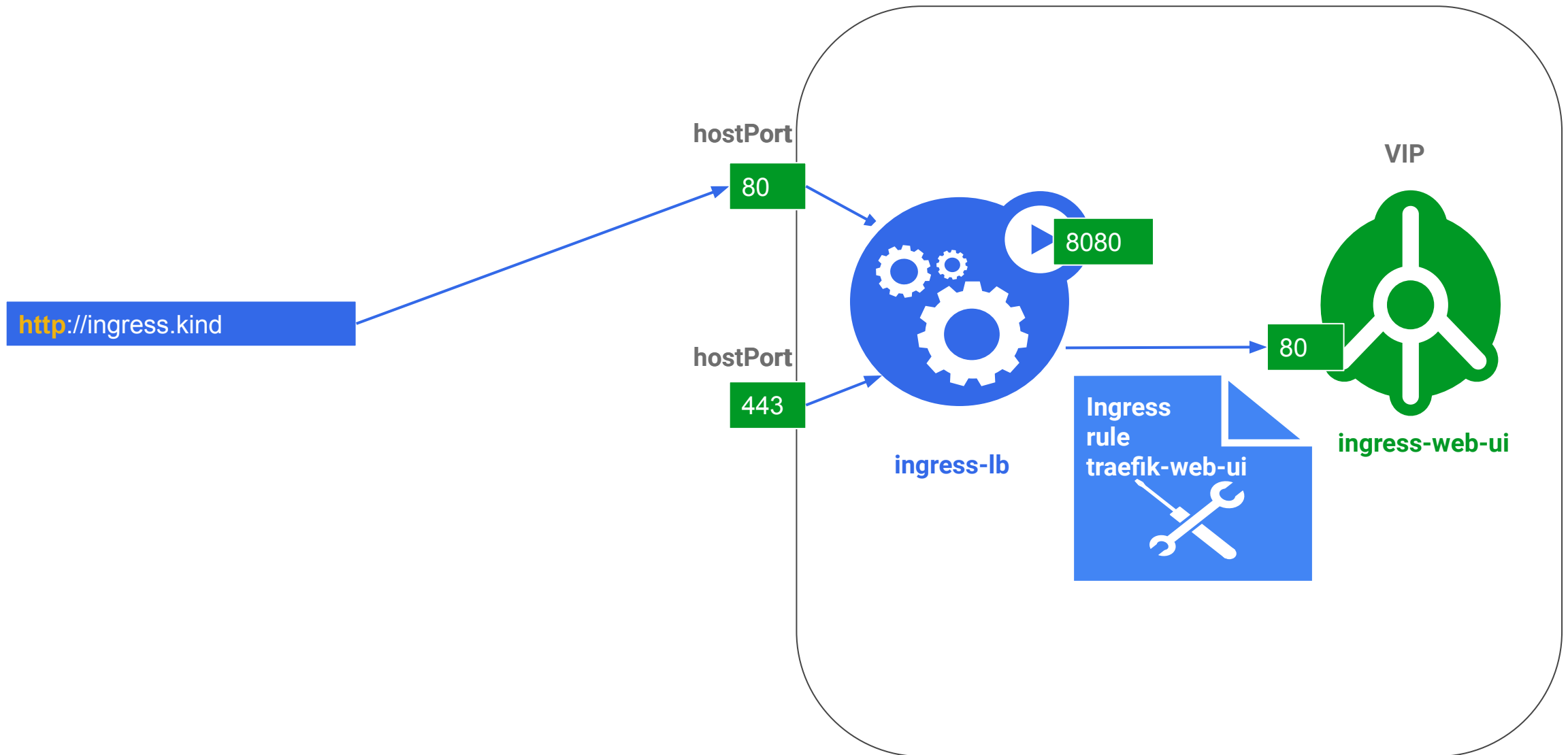
Traefik, daemonset mode



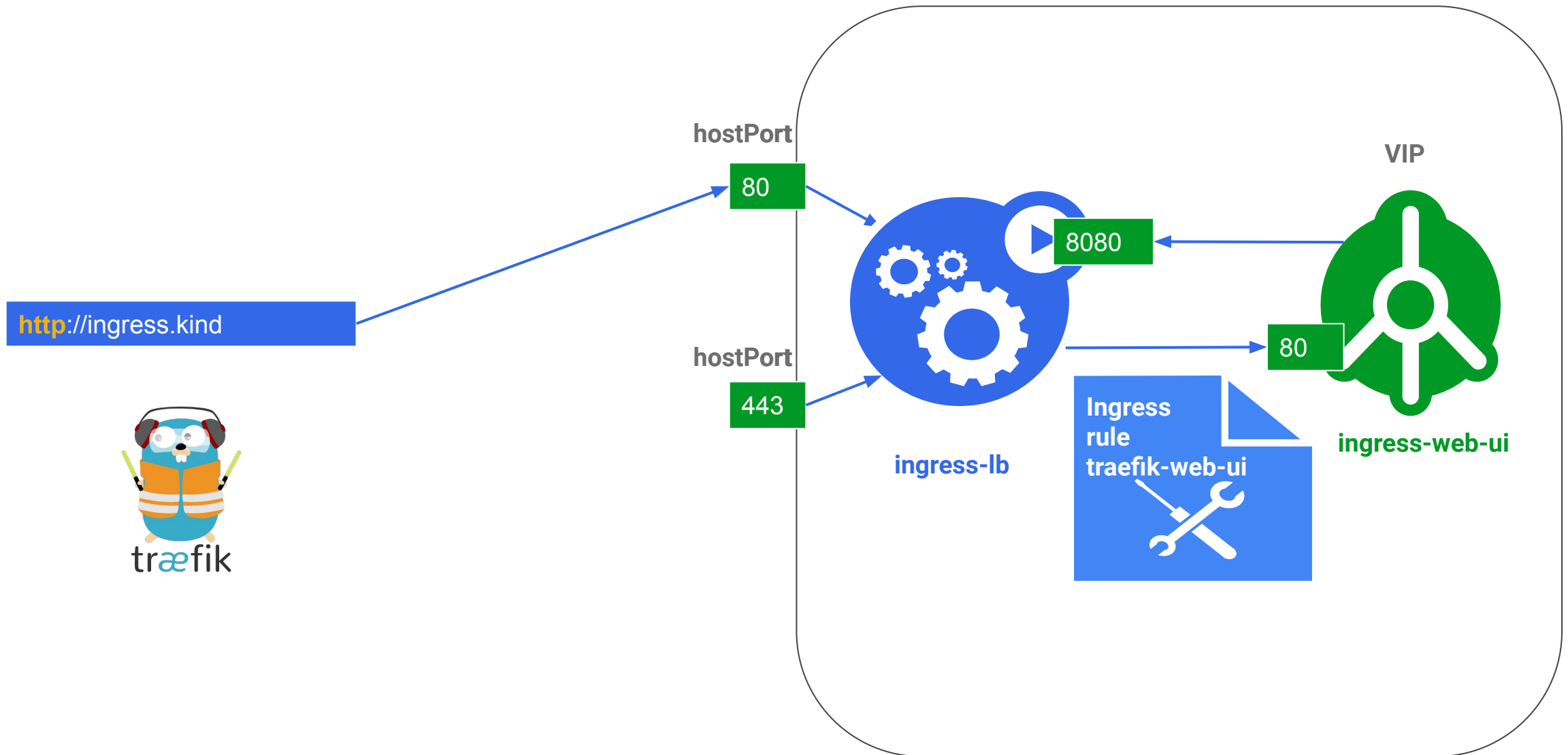
Traefik, daemonset mode



Traefik, daemonset mode



Traefik, daemonset mode



Questions?

Merci!

[@fjammes](#) on Github



<https://k8s-school.fr>